

## CSC343H5 Tutorial 1

# ERD and DDL Tutorial

In this session you will practise the two core skills of Checkpoint 1: drawing an Entity-Relationship Diagram (ERD) and translating it into PostgreSQL DDL. The priority is the ERD. If you do not reach the DDL questions today, that is fine. We will continue in the next tutorial.

## 1. Connecting to the Lab Database

All work is graded on the MCS lab machines. Verify that you can connect before the checkpoint deadline.

### On a lab machine

Open a terminal and run the following, replacing `yourutorid` with your actual UTORid in both places.

```
psql -h mcsdb.utm.utoronto.ca -U yourutorid -d yourutorid_343
```

Your password comes from your UToronto library barcode. Take the five digits starting at the seventh-last position. For example, if the last seven digits of your barcode are 2176101, the password is 21761.

### Connecting from off campus

You must connect to the UofT VPN before SSH will work. Open Cisco AnyConnect, connect to `general.vpn.utoronto.ca`, select group `UofT Default`, and sign in with your UTORid. Full instructions are at <https://isea.utoronto.ca/services/vpn/utorvpn/users/>.

Once on the VPN, SSH into any of the following servers and then run the `psql` command above.

```
ssh utorid@cs.utm.utoronto.ca
ssh utorid@cslinux.utm.utoronto.ca
ssh utorid@dh2026pc01.utm.utoronto.ca    % replace 2026 with your lab
room number
```

### Transferring files

You will need to get your `.ddl` files onto the server. You can use `scp` from the command line, or the VS Code Remote SSH extension, which lets you drag and drop files and provides SQL syntax highlighting. The VS Code setup guide is at <https://code.visualstudio.com/docs/remote/ssh>.

```
scp practical.ddl utorid@cs.utm.utoronto.ca:~/
```

### Useful psql commands

| Command                               | Effect                            |
|---------------------------------------|-----------------------------------|
| <code>\dt</code>                      | List all tables in your database  |
| <code>\d+ tablename</code>            | Describe the structure of a table |
| <code>\i filename.ddl</code>          | Run a DDL or SQL file             |
| <code>\q</code>                       | Quit psql                         |
| <code>SELECT * FROM tablename;</code> | Show all rows in a table          |

If you cannot connect, email Naaz at [naaz.sibia@utoronto.ca](mailto:naaz.sibia@utoronto.ca) as soon as possible. Do not wait until the deadline. Access issues are not grounds for an extension if reported late.

## 2. Domain Specification

You are modelling a small slice of a **Community Library System**. There are exactly three entities. Do not add any others.

| Entity          | Description                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------|
| <b>Member</b>   | A registered library member with a membership type.                                             |
| <b>Branch</b>   | A physical library branch, managed by one of its members.                                       |
| <b>BookCopy</b> | A physical copy of a book, owned by exactly one branch. Copies may be loaned to other branches. |

### 2.1 Member

Every member has a unique numeric identifier, a first name, a last name, and an email address that is unique across all members. A member has exactly one **membership\_type**, which is one of: **adult**, **student**, or **senior**. Every member records the date they joined the library.

A member must be affiliated with exactly one branch. A branch must have at least one affiliated member. The manager of a branch is always one of its members.

### 2.2 Branch

Every branch has a name (not globally unique), a street address, a city, an opening year, and a short specialty description. **A branch is uniquely identified by the combination of its name and its manager.** Do not assign a surrogate id to a branch.

*The manager relationship is a self-referential constraint within the Branch–Member relationship. Where else have we seen that..? (hint: project)*

### 2.3 BookCopy

Every book copy has a unique numeric identifier, a title, an ISBN (13 digits), an acquisition date, a condition (one of: **new**, **good**, **worn**, **damaged**), and a replacement cost in CAD greater than zero.

Each copy is owned by exactly one branch. A branch may own zero or more copies. Copies may be loaned to other branches; the loan records the start date and whether it is currently active. A copy may be on loan to multiple branches simultaneously. A branch cannot borrow its own copy.

## 3. Task 1: Entity-Relationship Diagram

Design an ERD for the domain above using **Chen notation**: entity rectangles, attribute ellipses with the primary key underlined, relationship diamonds, solid lines for total participation, and dashed lines for partial participation.

### Guiding questions

1. How do you show a composite primary key in Chen notation?
2. Which side of each relationship has total participation, and which has partial? Mark each correctly.

3. The inter-branch loan has its own attributes (`loan_start_date`, `is_active`). Where do relationship attributes appear in a Chen ERD?

#### 4. Task 2: PostgreSQL DDL

Translate your ERD into `CREATE TABLE` statements using the provided starter file `practical.ddl`. Run it with the command below.

```
psql -h mcsdb.utm.utoronto.ca -U yourutorid -d yourutorid_343 -f
practical.ddl
```

#### Naming conventions

| Element          | Convention                           | Example                        |
|------------------|--------------------------------------|--------------------------------|
| SQL keywords     | UPPERCASE                            | CREATE TABLE, NOT NULL         |
| Table names      | singular, snake_case                 | member, book_copy              |
| Primary key cols | table name plus _id                  | member_id                      |
| FK columns       | full PK name of the referenced table | member_id (referencing member) |
| Custom types     | snake_case                           | membership_type                |

#### Questions

Open `practical.ddl` and read through it before answering these. Most of the code is already written; the goal is to understand what it does and why.

1. Find the two `CREATE TYPE` statements near the top. What does the `ENUM` keyword do, and why is this more restrictive than just using `TEXT`?
2. The file creates `member` before `branch`, even though members belong to a branch. Read the comment explaining the circular dependency. In your own words, why can neither `member` nor `membership` rows be inserted first without a deferred constraint?
3. Look at how `book_copy` references `branch`. Why are there two FK columns (`owning_branch_name` and `owning_branch_manager_id`) instead of one?
4. The `membership` table has two blank `ON DELETE` actions that you need to fill in — one for the FK to `member`, one for the FK to `branch`. What are your options for each, and which makes the most sense?
5. `branch_loan` is the one table left for you to complete. Using `book_copy` directly above it as a guide, what columns and constraints does it need? Read the discussion question at the bottom of that table as a comment and think about it before you answer.

#### Before You Leave

The priority for today is the ERD. If you did not reach the DDL questions, that is completely fine, we will continue in the next tutorial, and you are welcome to work through them on your own in the meantime. This exercise starts early to give you time to get familiar with the domain before Assignment 1 is due.

**Checklist**

1. I have confirmed I can connect to the lab database, or I have emailed Naaz.
2. I can draw an entity with attribute ellipses and an underlined primary key.
3. I understand total versus partial participation and when to use a solid line.
4. I can draw a relationship diamond with cardinality labels on both sides.
5. I know why the branch manager is a relationship, not an attribute ellipse on Branch.
6. I understand what a composite primary key is and how it appears in an ERD.
7. I know that some constraints cannot be expressed in an ERD or in basic DDL.